

*Examen Final (CF) — Corrigé officiel***ESIN : Semestre 6**

Web Development I | Saad NOUFEL | Mai 2026

DOCUMENT RÉSERVÉ AU CORRECTEUR

Barème total : 20 points Ex.1 Sessions & Cookies : 5 pts Ex.2 PDO : 6 pts Ex.3 POO : 9 pts
QCM : 5 pts (1 pt/question, réponses partielles = 0)

Exercice 1 Sessions & Cookies /5 ptsSystème d'authentification pour **CampusMedia** — vidéothèque universitaire.**Question 1****2 pts****Corrigé Q1 — connexion.php**

```
1 <?php
2 session_start();
3 require_once 'connexion.php';
4
5 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
6     $login = trim($_POST['login'] ?? '');
7     $mdp = trim($_POST['mot_de_passe'] ?? '');
8
9     if (empty($login) || empty($mdp)) {
10         echo "Veuillez remplir tous les champs.";
11     } else {
12         $pdo = getConnexion();
13         $stmt = $pdo->prepare(
14             "SELECT * FROM utilisateurs WHERE login = ?"
15         );
16         $stmt->execute([$login]);
17         $user = $stmt->fetch(PDO::FETCH_ASSOC);
18
19         if ($user && password_verify($mdp, $user['mot_de_passe'])) {
20             $_SESSION['utilisateur'] = $user['login'];
21             $_SESSION['role'] = $user['role'] ?? 'visiteur';
22
23             setcookie(
24                 'derniere_connexion',
25                 date('d/m/Y H:i'),
26                 time() + 30 * 24 * 3600,
27                 '/'
28             );
29             header('Location: catalogue.php');
30             exit;
31         } else {
32             echo "Identifiants incorrects.";
33         }
34     }
35 }
```

```
36 ?>
```

Points clés à vérifier (2 pts) :

- 0.25 pt — `session_start()` en tout premier
- 0.25 pt — Vérification `empty()` sur les deux champs
- 0.25 pt — Prepared statement correct avec `?` ou `:login`
- 0.25 pt — `password_verify()` utilisé (pas comparaison directe)
- 0.5 pt — `$_SESSION` rempli avec login ET rôle
- 0.25 pt — `setcookie()` avec expiration 30 jours (`time()+30*24*3600`)
- 0.25 pt — `header('Location: ...')` suivi de `exit`

Question 2**1.5 pts****Corrigé Q2 — début de catalogue.php**

```
1 <?php
2 session_start();
3
4 if (!isset($_SESSION['utilisateur'])) {
5     header('Location: connexion.php');
6     exit;
7 }
8
9 $login = htmlspecialchars($_SESSION['utilisateur']);
10 $role = htmlspecialchars($_SESSION['role'] ?? 'visiteur');
11
12 echo "Bienvenue, $login ! (role : $role)";
13
14 if (isset($_COOKIE['derniere_connexion'])) {
15     $date = htmlspecialchars($_COOKIE['derniere_connexion']);
16     echo "Derniere connexion : $date";
17 } else {
18     echo "Premiere connexion";
19 }
20 ?>
```

Points clés (1.5 pt) :

- 0.5 pt — Guard clause : `!isset($_SESSION['utilisateur'])` + redirect + `exit`
 - 0.5 pt — Affichage login et rôle depuis `$_SESSION`
 - 0.5 pt — `isset($_COOKIE[...])` avec les deux cas (connexion / première fois)
- Accepter sans `htmlspecialchars()` (non demandé explicitement mais apprécié).

Question 3**1.5 pts****Corrigé Q3 — deconnexion.php**

```
1 <?php
2 session_start();
3 session_unset();
4 session_destroy();
5
6 setcookie('derniere_connexion', '', time() - 3600, '/');
7
```

```
8 header('Location: connexion.php');
9 exit;
10 ?>
```

Points clés (1.5 pt) :

- **0.5 pt** — `session_unset()` ET `session_destroy()` (les deux)
- **0.5 pt** — Suppression cookie : `setcookie('...', "", time()-3600, '/')`
- **0.5 pt** — Redirection + `exit`

Remarque : `session_unset()` seul sans `session_destroy()` = -0.25 pt.

Exercice 2 PHP et PDO /6 ptsGestion du catalogue de films **CampusMedia** via PDO — table FILMS.**Question 1****1 pts****Corrigé Q1 — getConnexion()**

```
1 function getConnexion(): PDO {
2     $dsn = "mysql:host=localhost;dbname=campusmedia_bd;charset=utf8mb4"
3     ;
4     return new PDO($dsn, 'mediauser', 'media2026', [
5         PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
6         PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
7     ]);
8 }
```

Points clés (1 pt) :

- **0.25 pt** — DSN correct (mysql:host=...;dbname=...)
- **0.25 pt** — Identifiants corrects passés au constructeur PDO
- **0.25 pt** — PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
- **0.25 pt** — PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC

Question 2**2 pts****Corrigé Q2 — getAll()**

```
1 public static function getAll(PDO $pdo, int $limit = 0): array {
2     $sql = "SELECT * FROM FILMS ORDER BY note DESC";
3     if ($limit > 0) {
4         $sql .= " LIMIT " . (int) $limit;
5     }
6     $stmt = $pdo->query($sql);
7     $films = [];
8     while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
9         $films[] = new Film(
10             $row['titre'],
11             $row['realisateur'],
12             $row['genre'],
13             (int) $row['annee'],
14             (float) $row['note'],
15             (int) $row['id']
16         );
17     }
18     return $films;
19 }
```

Points clés (2 pts) :

- **0.5 pt** — ORDER BY note DESC présent
- **0.5 pt** — Gestion du \$limit avec cast (int) pour sécurité
- **0.5 pt** — Boucle while/fetchAll et construction des objets Film
- **0.5 pt** — Retourne le tableau \$films

Accepter fetchAll() + foreach à la place de while + fetch().

Question 3

1.5 pts

Corrigé Q3 — save()

```
1 public function save(PDO $pdo): bool {
2     $sql = "INSERT INTO FILMS (titre, realisateur, genre, annee, note)
3         VALUES (:titre, :realisateur, :genre, :annee, :note)";
4     $stmt = $pdo->prepare($sql);
5     $stmt->bindValue(':titre', $this->titre, PDO::PARAM_STR);
6     $stmt->bindValue(':realisateur', $this->realisateur, PDO::PARAM_STR);
7     $stmt->bindValue(':genre', $this->genre, PDO::PARAM_STR);
8     $stmt->bindValue(':annee', $this->annee, PDO::PARAM_INT);
9     $stmt->bindValue(':note', $this->note);
10    $ok = $stmt->execute();
11    if ($ok) {
12        $this->id = (int) $pdo->lastInsertId();
13    }
14    return $ok;
15 }
```

Points clés (1.5 pt) :

- 0.5 pt — prepare() avec paramètres nommés :label (pas de concaténation)
- 0.5 pt — bindValue() ou execute([...]) correct pour les 5 champs
- 0.5 pt — \$this->id = (int) \$pdo->lastInsertId() après succès

Remarque : DECIMAL (note) n'a pas de constante PDO::PARAM_* spécifique — laisser sans type est correct.

Question 4

1.5 pts

Corrigé Q4 — findByGenre()

```
1 public static function findByGenre(PDO $pdo, string $genre): array {
2     $stmt = $pdo->prepare(
3         "SELECT * FROM FILMS WHERE genre = ? ORDER BY note DESC"
4     );
5     $stmt->execute([$genre]);
6     return $stmt->fetchAll(PDO::FETCH_ASSOC);
7 }
```

Explication : Le *prepared statement* sépare le code SQL des données : la valeur de `$genre` est transmise comme paramètre lié, jamais interprétée comme du SQL. La chaîne malveillante ' OR '1'='1 est traitée comme une valeur littérale et ne modifie pas la structure de la requête.

Points clés (1.5 pt) :

- 0.5 pt — prepare() avec ? ou :genre (jamais concaténation)
- 0.5 pt — execute([\$genre]) et fetchAll()
- 0.5 pt — Explication correcte de la protection anti-injection SQL

Exercice 3 POO en PHP /9 ptsModélisation des ressources de la vidéothèque **CampusMedia**.**Question 1****2 pts****Corrigé Q1 — Classe abstraite Ressource**

```
1 abstract class Ressource {
2     protected int $id;
3     protected string $titre;
4     protected string $dateAjout;
5
6     public function __construct(
7         int $id, string $titre, string $dateAjout
8     ) {
9         $this->id = $id;
10        $this->titre = $titre;
11        $this->dateAjout = $dateAjout;
12    }
13
14    abstract public function getType(): string;
15
16    public function __toString(): string {
17        return "[{$this->id}] {$this->titre} | Ajoute le: {$this->
18            dateAjout}";
19    }
20 }
```

Points clés (2 pts) :

- 0.5 pt — Mot-clé `abstract` sur la classe
- 0.5 pt — Les trois attributs `protected` avec types corrects
- 0.5 pt — Méthode `abstract public function getType(): string` (sans corps)
- 0.5 pt — `__toString()` concrète affichant les 3 attributs

Question 2**1 pts****Corrigé Q2 — Interface Empruntable**

```
1 interface Empruntable {
2     public function emprunter(string $nomEtudiant): bool;
3     public function retourner(): bool;
4     public function getEmprunteur(): ?string;
5 }
```

Points clés (1 pt) :

- 0.25 pt — Mot-clé `interface`
- 0.25 pt — `emprunter(string $nomEtudiant): bool` (signatures exactes)
- 0.25 pt — `retourner(): bool`
- 0.25 pt — `getEmprunteur(): ?string` (nullable string)

Question 3**6 pts**

Corrigé Q3 — Classe Film (1/2) : structure, constructeur, getter, setter, getType()

```

1 class Film extends Ressource implements Empruntable {
2
3     private string $realisateur;
4     private string $genre;
5     private int $dureeMinutes;
6     private bool $estDisponible;
7     private ?string $emprunteur;
8
9     public function __construct(
10         int $id, string $titre, string $dateAjout,
11         string $realisateur, string $genre, int $dureeMinutes
12     ) {
13         parent::__construct($id, $titre, $dateAjout);
14         $this->realisateur = $realisateur;
15         $this->genre = $genre;
16         $this->dureeMinutes = $dureeMinutes;
17         $this->estDisponible = true;
18         $this->emprunteur = null;
19     }
20
21     // Getter
22     public function getRealisateur(): string {
23         return $this->realisateur;
24     }
25
26     // Setter
27     public function setGenre(string $genre): void {
28         $this->genre = $genre;
29     }
30
31     // Implementation de la methode abstraite
32     public function getType(): string {
33         return "Film";
34     }

```

Corrigé Q3 — Classe Film (2/2) : Empruntable + __toString()

```

1 // --- Implementation : Interface Empruntable ---
2
3 public function emprunter(string $nomEtudiant): bool {
4     if (!$this->estDisponible) {
5         return false;
6     }
7     $this->estDisponible = false;
8     $this->emprunteur = $nomEtudiant;
9     return true;
10 }
11
12 public function retourner(): bool {
13     if ($this->estDisponible) {
14         return false;
15     }
16     $this->estDisponible = true;
17     $this->emprunteur = null;
18     return true;
19 }

```

```
20
21     public function getEmprunteur(): ?string {
22         return $this->emprunteur;
23     }
24
25     // --- Surcharge de __toString() ---
26
27     public function __toString(): string {
28         $dispo = $this->estDisponible
29             ? "Disponible"
30             : "Emprunte par " . $this->emprunteur;
31         return parent::__toString()
32             . " | Type : " . $this->getType()
33             . " | Realisateur : " . $this->realisateur
34             . " | Genre : " . $this->genre
35             . " | Duree : " . $this->dureeMinutes . " min"
36             . " | Statut : " . $dispo;
37     }
38 }
```

Points clés (6 pts) :

- 0.5 pt — class Film extends Ressource implements Empruntable
- 0.5 pt — Les 5 attributs propres avec types corrects (?string pour emprunteur)
- 1 pt — Constructeur : appel parent::__construct() + init attributs propres avec valeurs par défaut
- 0.5 pt — Getter getRealisateur() et setter setGenre()
- 0.5 pt — getType() retournant "Film"
- 1 pt — emprunter() : vérification estDisponible avant d'emprunter
- 1 pt — retourner() : vérification que le film est bien emprunté
- 0.5 pt — getEmprunteur() retournant ?string
- 0.5 pt — __toString() appelant parent::__toString() et ajoutant les attributs propres

QCM — Corrigé /5 pts (1 pt par question, réponse complète uniquement)

Les cases ☒ indiquent les **bonnes réponses**. Les cases ☐ indiquent les mauvaises réponses.
Une question avec plusieurs bonnes réponses : toutes doivent être cochées ET aucune mauvaise cochée pour obtenir le point.

Question 1 (1 pt) : Laquelle des affirmations est correcte concernant `session_start()` ?

- ☐ Elle peut être appelée après l'envoi de contenu HTML
- ☒ Elle doit être appelée avant tout output, en tout premier
- ☒ Elle crée automatiquement un cookie côté client avec l'ID de session
- ☐ Elle stocke les données de session dans le navigateur

Justification : `session_start()` envoie un header HTTP (`Set-Cookie: PHPSESSID=...`) — impossible après du HTML. Elle crée bien un cookie avec l'ID de session. Les données elles-mêmes sont stockées **côté serveur** (dans `/tmp/sess_XXX`).

Question 2 (1 pt) : Comment supprimer définitivement un cookie nommé `token` en PHP ?

- ☐ `unset($_COOKIE['token'])`
- ☒ `setcookie('token', '', time() - 3600, '/')`
- ☐ `deletetecookie('token')`
- ☐ `session_destroy()`

Justification : `unset($_COOKIE[...])` supprime la variable PHP locale mais **pas** le cookie dans le navigateur. La seule façon est de renvoyer `setcookie()` avec une date d'expiration dans le passé. `deletetecookie()` n'existe pas en PHP.

Question 3 (1 pt) : Parmi les instructions PDO, laquelle protège contre les injections SQL ?

- ☐ `$pdo->query("SELECT * FROM films WHERE genre = '$genre'");`
- ☒ `$stmt = $pdo->prepare("SELECT * FROM films WHERE genre = ?"); $stmt->execute([$genre]);`
- ☐ `$pdo->exec("SELECT * FROM films WHERE genre = " . $genre);`
- ☐ `$pdo->query("SELECT * FROM films WHERE genre = " . htmlspecialchars($genre));`

Justification : Seul le *prepared statement* sépare code SQL et données. `htmlspecialchars()` protège contre XSS, pas contre SQL injection.

Question 4 (1 pt) : Lesquelles des affirmations sur les classes abstraites et interfaces sont vraies ?

- ☒ Une classe abstraite peut contenir des méthodes avec implémentation
- ☐ Une interface peut contenir des méthodes avec implémentation (corps)
- ☐ Une classe peut hériter de plusieurs classes abstraites à la fois en PHP
- ☒ Une classe peut implémenter plusieurs interfaces à la fois en PHP

Justification : En PHP, une classe abstraite peut avoir des méthodes concrètes ET abstraites. Une interface ne contient que des signatures (pas de corps, sauf `default` dans d'autres langages). PHP n'a pas d'héritage multiple pour les classes. Une classe peut implémenter plusieurs interfaces : `class X implements A, B, C.`

Question 5 (1 pt) : Que retourne `$stmt->fetch(PDO::FETCH_ASSOC)` quand aucun résultat ?

- ☐ `[]`

- ☐ `null`
- ☒ `false`
- ☐ Une exception `PDOException`

Justification : La documentation PHP précise que `fetch()` retourne `false` s'il n'y a plus de ligne à récupérer. C'est pourquoi on écrit `if (!$row = $stmt->fetch())` ou `if ($row === false)`.

Partie	Barème	Note obtenue
Ex.1 Q1 — connexion.php	/2	
Ex.1 Q2 — catalogue.php (début)	/1.5	
Ex.1 Q3 — deconnexion.php	/1.5	
Exercice 1 total	/5	
Ex.2 Q1 — getConnexion()	/1	
Ex.2 Q2 — getAll()	/2	
Ex.2 Q3 — save()	/1.5	
Ex.2 Q4 — findByGenre()	/1.5	
Exercice 2 total	/6	
Ex.3 Q1 — Classe abstraite	/2	
Ex.3 Q2 — Interface	/1	
Ex.3 Q3 — Classe Film	/6	
Exercice 3 total	/9	
QCM (5 questions × 1 pt)	/5	
TOTAL	/20	

— Fin du corrigé —
